CHAPTER 1

Syllabus Map & Strategy

GATE 2027 COA syllabus · what to study · priority order · how these notes are organised

QUICK TRICK

The whole GATE 2027 COA syllabus is **six lines**: Instruction set & addressing modes · ALU design · Control unit (hardwired / microprogrammed) · Memory interfacing & hierarchy (performance, cache mapping) · I/O (interrupt, DMA) · Pipelining & hazards.
Roughly **8–10 marks** per paper (typical, not official). Almost all of it is **calculation**: practise numbers.

1.1 The official syllabus (GATE 2027, IIT Madras)

● **LEVEL 1 · BEGINNER**

Section 3 of the CS paper, word for word from the official syllabus PDF (gate2027.iitm.ac.in):

```
Instruction set and addressing modes.
Design of arithmetic and logic unit (ALU).
Design of control unit - hardwired and microprogrammed.
Memory interfacing and hierarchy: performance, cache memory mapping.
I/O interface (interrupt and DMA).
Instruction pipelining, pipeline hazards.
```

Also in the official file, but under **Section 2 (Digital Logic)**: "Number representation and arithmetic (fixed and floating point)". COA questions use it constantly, so Chapter 3 covers it here. If you already have it in your Digital Logic notes, use Chapter 3 as revision.

Always confirm the final wording on the official site before the exam. Earlier years used a differently worded COA list (for example "memory hierarchy: cache, main memory and secondary storage"), so disk topics are not named now. They are kept as a short Chapter 11 because they are cheap marks if asked.

1.2 Where the marks come from

● **LEVEL 2 · EXAM**

This tier list is my own estimate from the usual pattern of GATE papers, not an official statement. Use it only to order your time.

Tier	Topics	Notes chapter
Tier 1 (very high)	Pipelining (speedup, stalls, hazards) · cache mapping and cache performance · addressing modes and instruction formats	8 · 10 · 4, 5
Tier 2 (high)	IEEE 754 and number representation · CPU performance (CPI, Amdahl) · memory performance and chip interfacing · DMA and interrupt calculations · microprogrammed control word size	3 · 2 · 9 · 12 · 7
Tier 3	ALU (Booth, adders) · disk access time · hardwired control	6 · 11 · 7

GATE POINT

Most 2-mark COA questions are **numerical answer type (NAT)** or multi-step cache/pipeline problems. There is no negative marking for NAT and MSQ, so always attempt.

1.3 Chapter map of these notes

● LEVEL 1 · BEGINNER

Ch	Title	Syllabus clause covered
2	CPU Performance	Performance (CPI, MIPS, Amdahl)
3	Number Representation & Arithmetic	Digital Logic section: fixed and floating point
4	Instruction Set & Formats	Instruction set
5	Addressing Modes	Addressing modes
6	ALU Design	Design of ALU
7	Control Unit: Hardwired & Microprogrammed	Design of control unit
8	Instruction Pipelining & Hazards	Pipelining, hazards
9	Memory Hierarchy, Performance & Interfacing	Memory interfacing and hierarchy: performance
10	Cache Memory Mapping	Cache memory mapping
11	Disk & Secondary Storage	Memory hierarchy (older wording)
12	I/O: Interrupt & DMA	I/O interface (interrupt and DMA)
13	Master Formula Sheet & Mixed Practice	Revision

1.4 Study order and strategy

● LEVEL 2 · EXAM

1. CPU performance (Ch 2) → 2. Pipelining (Ch 8) → 3. Cache (Ch 9, 10) → 4. ISA and addressing modes (Ch 4, 5) → 5. Number representation (Ch 3) → 6. I/O (Ch 12) → 7. Control unit and ALU (Ch 7, 6) → 8. Disk (Ch 11).

COMMON MISTAKE

- Treating COA as theory. Practise with units (bits vs bytes, KB = 2^{10}, ns vs cycles).
- Mixing "words" and "bytes" when counting address bits.
- Skipping the number-representation chapter because it sits in Digital Logic.

1.5 Books

Hamacher (Computer Organization) · Patterson & Hennessy (Computer Organization and Design; Computer Architecture) · Morris Mano (Computer System Architecture). Solve previous-year GATE questions after each chapter.

Next: Chapter 2 — CPU Performance (CPI, MIPS, Amdahl's law).

CHAPTER 2

CPU Performance

CPU time · CPI · clock rate · MIPS · weighted CPI · Amdahl's law · speedup

QUICK TRICK

CPU time = IC × CPI × T_{clk} = IC × CPI / f. Weighted CPI = Σ (fraction$_i$ × CPI$_i$). **Speedup = old time ÷ new time**. Amdahl: **S = 1 / [(1 − f) + f / k]**.

2.1 What does "faster" mean?

● **LEVEL 1 · BEGINNER**

The problem. Which computer is better? **The idea.** Compare the **time** to run the same program. **Analogy.** Two cars on one road: time taken is what counts, not how loud the engine is.

- **Latency (response time):** time for one task. **Throughput:** tasks per unit time.
- **Clock cycle time** T_{clk} = 1/f. At 2 GHz, T_{clk} = 0.5 ns.

2.2 The CPU time equation

● **LEVEL 2 · EXAM**

```
CPU time = Instruction count (IC) x Cycles per instruction (CPI) x Clock cycle time
         = IC x CPI / f
Total cycles = IC x CPI          MIPS = f / (CPI x 10^6) = IC / (CPU time x 10^6)
```

Factor	Changed by
IC	Compiler, instruction set
CPI	Organisation (pipeline, cache), instruction mix
T_{clk} (or f)	Technology, organisation

Weighted CPI. With instruction classes of fractions F_i and CPI$_i$: CPI = Σ F_i × CPI$_i$. **Example.** 50% ALU (CPI 1), 30% load/store (CPI 2), 20% branch (CPI 3): CPI = 0.5 + 0.6 + 0.6 = **1.7**.

Example. IC = 10^6, CPI = 2.5, f = 500 MHz: time = 10^6 × 2.5 / (5×10^8) = **5 ms**.

2.3 Amdahl's law

● **LEVEL 2 · EXAM**

If a fraction **f** of the execution time is made **k times faster**, the rest stays the same:

```
New time = (1 - f) + f/k            Speedup = 1 / [ (1 - f) + f/k ]
Maximum speedup (k -> infinity) = 1 / (1 - f)
```

Example. f = 0.8, k = 4: speedup = 1/(0.2 + 0.2) = **2.5**. Even with infinite k the limit is 1/0.2 = 5. **Two parts improved:** 40% by 2× and 30% by 3×: new time = 0.3 + 0.2 + 0.1 = 0.6, speedup = **1.67**.

QUICK TRICK

Amdahl uses fractions of **time before the improvement**, not fractions of instructions.

2.4 MIPS and benchmark means

● **LEVEL 3 · GATE**

- **MIPS** ignores what each instruction does, so it cannot compare different instruction sets (a CISC instruction does more than a RISC one).
- Averages: **arithmetic mean** for execution times, **harmonic mean** for rates (MIPS, MFLOPS), **geometric mean** for normalised ratios.
- Comparing machines: speedup of A over B = T_B / T_A.

2.5 Common mistakes

COMMON MISTAKE

- Using the clock rate alone to compare machines (CPI and IC matter too).
- Forgetting the unit: MHz = 10^6, GHz = 10^9, ns = 10^{-9} s.
- Applying Amdahl with f as a fraction of the **new** time.
- Averaging CPI values without weighting by instruction fractions.

GATE QUICK CHECK — Chapter 2

```
T = IC*CPI/f      CPI = sum(Fi*CPIi)      MIPS = f/(CPI*10^6)
Speedup = Told/Tnew      Amdahl = 1/((1-f)+f/k)      max = 1/(1-f)
IC x 0.8 and CPI x 1.1  ->  time x 0.88
```

Practice questions

● **LEVEL 1 · BEGINNER**

Q1. Clock cycle time of a 2 GHz processor.

Q2. A program has 10^6 instructions, CPI 2.5, clock 500 MHz. CPU time?

Q3. Mix: 50% ALU (CPI 1), 30% load/store (CPI 2), 20% branch (CPI 3). Average CPI?

Q4. Why is MIPS a poor way to compare computers?

● **LEVEL 2 · EXAM**

Q5. 80% of a program is sped up 4×. Overall speedup? Maximum possible if that part became infinitely fast?

Q6. CPI = 1.7 on a 1 GHz machine. MIPS rating?

Q7. Machine A: CPI 2 at 1 GHz. Machine B: CPI 1.5 at 700 MHz. Same program. Which is faster and by how much?

Q8. A compiler change cuts IC by 20% but raises CPI by 10% (same clock). Speedup?

● **LEVEL 3 · GATE**

Q9. A component taking 40% of the time is made 2× faster and another taking 30% is made 3× faster. Overall speedup?

Q10. Program of 2×10^9 instructions runs in 4 s on a 2 GHz machine. Find CPI and MIPS.

Q11. Floating-point operations are 25% of the time. A new FP unit makes them 5× faster. Overall speedup?

Answers

Q	Solution
1	$1/(2\times10^9)$ = **0.5 ns**.
2	$10^6 \times 2.5 / 5\times10^8$ = **5 ms**.
3	0.5 + 0.6 + 0.6 = **1.7**.
4	Different instruction sets do different amounts of work per instruction.
5	1/(0.2 + 0.8/4) = **2.5**. Maximum = 1/0.2 = **5**.
6	1000/1.7 ≈ **588 MIPS**.
7	Per instruction: A = 2 ns, B = 1.5/0.7 = 2.143 ns. **A is faster** by 2.143/2 ≈ **1.07×**.
8	New time = 0.8 × 1.1 = 0.88 of old. Speedup = 1/0.88 ≈ **1.14**.
9	New time = 0.4/2 + 0.3/3 + 0.3 = 0.2 + 0.1 + 0.3 = 0.6. Speedup = **1.67**.
10	Cycles = $4 \times 2\times10^9 = 8\times10^9$; CPI = $8\times10^9/2\times10^9$ = **4**. MIPS = $2\times10^9/(4 \times 10^6)$ = **500**.
11	1/(0.75 + 0.25/5) = 1/0.8 = **1.25**.

Next: Chapter 3 — Number Representation and Arithmetic (2's complement, IEEE 754).

AMJESH
singhamjesh.github.io

CHAPTER 3

Number Representation & Arithmetic

Unsigned · sign-magnitude · 1's and 2's complement · overflow · IEEE 754 single and double · special values

QUICK TRICK

n-bit **2's complement** range: $\mathbf{-2^{n-1} \ldots 2^{n-1} - 1}$. Negate = **invert + 1**. Overflow = carry into MSB ≠ carry out of MSB. IEEE single: **1 | 8 | 23**, bias **127**; double: **1 | 11 | 52**, bias **1023**. Value = $(-1)^s \times 1.f \times 2^{e - bias}$.

(This topic is listed under Digital Logic in the official syllabus; it is placed here because COA uses it in ALU and data-path questions.)

3.1 Integer representations

● **LEVEL 1 · BEGINNER**

Scheme	n-bit range	Zeros	Negate
Unsigned	$0 \ldots 2^n - 1$	1	–
Sign-magnitude	$-(2^{n-1} - 1) \ldots +(2^{n-1} - 1)$	2 (+0, −0)	flip the sign bit
1's complement	$-(2^{n-1} - 1) \ldots +(2^{n-1} - 1)$	2	invert all bits
2's complement	$\mathbf{-2^{n-1} \ldots 2^{n-1} - 1}$	1	invert, then add 1

- **2's complement value** = $-b_{n-1} \times 2^{n-1}$ + (the other bits as usual). 8-bit 0x9C = 10011100 = −128 + 16 + 8 + 4 = **−100**.
- **Sign extension:** to widen a 2's complement number, copy the sign bit into the new high bits.
- **Excess-K (biased):** stored = value + K. Used for floating-point exponents.
- The 2's complement range has **one more negative** number than positive (the most negative has no positive partner).

Example. −25 in 8 bits: 25 = 00011001 → invert 11100110 → +1 = **11100111**.

3.2 Addition, subtraction, overflow

● **LEVEL 2 · EXAM**

- Subtract A − B as **A + (−B)** = A + B' + 1. The same adder does both.
- **Overflow** (signed): adding two positives gives a negative, or two negatives give a positive. Test: **V = C_{in} to MSB XOR C_{out} of MSB**.
- **Carry out** (C flag) signals overflow for **unsigned** numbers. Adding numbers of opposite sign never overflows.

Example. 8-bit 0x7F + 0x01 = 0x80: two positives gave a negative, so **overflow**. 4-bit 0110 + 0101 = 1011: carry into MSB = 1, carry out = 0, so V = 1.

3.3 IEEE 754 floating point

● **LEVEL 2 · EXAM**

```
Single (32 bit):  | s (1) | exponent e (8) | fraction f (23) |   bias 127
Double (64 bit):  | s (1) | exponent e (11)| fraction f (52) |   bias 1023
Normal number  : value = (-1)^s x 1.f x 2^(e - bias)       (hidden leading 1)
```

Exponent e	Fraction f	Meaning (single)
0	0	±0
0	≠ 0	Denormal: $(-1)^s \times 0.f \times 2^{-126}$
1 ... 254	any	Normal number
255	0	±Infinity

Exponent e	Fraction f	Meaning (single)
255	≠ 0	NaN

Encode 5.75. $5.75 = 101.11_2 = 1.0111 \times 2^2$. e = 2 + 127 = 129 = 10000001. Bits: 0 | 10000001 | 0111000...0 → **0x40B80000**.

Encode −0.15625. $0.15625 = 0.00101_2 = 1.01 \times 2^{-3}$. e = 124 = 01111100. Bits: 1 | 01111100 | 0100...0 → **0xBE200000**.

Decode 0xC1200000. 1 | 10000010 | 0100...0: sign −, e = 130 → exponent 3, mantissa 1.01 = 1.25. Value = $-1.25 \times 8 = \mathbf{-10.0}$.

Quantity (single)	Value
Largest finite	$(2 - 2^{-23}) \times 2^{127} \approx 3.4 \times 10^{38}$
Smallest positive normal	2^{-126}
Smallest positive denormal	2^{-149}
Precision	24 bits (23 + hidden 1) ≈ 7 decimal digits
NaN patterns	$2 \times (2^{23} - 1) = 2^{24} - 2$

3.4 Floating-point addition (steps)

● LEVEL 3 · GATE

- **Align** exponents: shift the mantissa of the smaller number right by the difference.
- **Add** (or subtract) the mantissas.
- **Normalise** the result to 1.f form (adjust exponent).
- **Round**, and check overflow/underflow.

QUICK TRICK

Numbers such as 0.1 cannot be stored exactly in binary, so floating-point results carry small rounding errors. Between 2^k and 2^{k+1} there are exactly $\mathbf{2^{23}}$ single-precision values.

3.5 Common mistakes

COMMON MISTAKE

- Forgetting the hidden 1 (the mantissa is 1.f, not 0.f, for normal numbers).
- Using bias 128 instead of 127 (or 1024 instead of 1023).
- Treating the carry flag as signed overflow.
- Taking the 2's complement range as symmetric.

GATE QUICK CHECK — Chapter 3

```
2s compl: -2^(n-1)..2^(n-1)-1     negate = invert+1     V = C_into_MSB XOR C_out_MSB
single 1|8|23 bias127   double 1|11|52 bias1023     value = (-1)^s * 1.f * 2^(e-bias)
e=0,f=0 zero | e=0,f!=0 denormal | e=max,f=0 inf | e=max,f!=0 NaN
5.75 = 0x40B80000     -10.0 = 0xC1200000     0.75 = 0x3F400000
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Range of an 8-bit 2's complement number.

Q2. Write −25 as an 8-bit 2's complement number.

Q3. Decimal value of the 8-bit 2's complement number 0x9C.

Q4. Exponent bias for IEEE single and double precision.

● LEVEL 2 · EXAM

Q5. 8-bit 2's complement: 0x7F + 0x01. Result and flags?

Q6. Decode the single-precision number 0xC1200000.

Q7. Encode 0.75 in IEEE single precision (hex).

Q8. How many distinct 32-bit patterns represent NaN?

● **LEVEL 3 · GATE**

Q9. Smallest positive normal and denormal single-precision numbers.

Q10. In 4-bit 2's complement compute 0110 + 0101. Carry into and out of the MSB, and the overflow flag.

Q11. How many 8-bit patterns exist in 1's complement for zero, and what is the range?

Q12. How many single-precision numbers lie in the interval [1, 2)?

Answers

Q	Solution
1	**−128 … +127**.
2	**11100111**.
3	156 − 256 = **−100**.
4	**127** and **1023**.
5	Result 0x80 (−128). Signed **overflow** (two positives gave a negative); no carry out.
6	**−10.0** (see 3.3).
7	$0.75 = 1.1 \times 2^{-1}$; e = 126 = 01111110. Bits 0 | 01111110 | 1000…0 = **0x3F400000**.
8	e = 255, f ≠ 0: $2 \times (2^{23} - 1)$ = **16,777,214**.
9	$\mathbf{2^{-126}}$ and $\mathbf{2^{-149}}$.
10	Sum 1011. Carry into MSB = 1, carry out = 0, so **V = 1** (overflow: 6 + 5 = 11 > 7).
11	Two zeros (00000000, 11111111). Range **−127 … +127**.
12	$\mathbf{2^{23}}$ **= 8,388,608** (exponent fixed, 23 fraction bits free).

Next: Chapter 4 — Instruction Set and Instruction Formats.

CHAPTER 4

Instruction Set & Formats

Instruction parts · 0/1/2/3-address machines · RISC vs CISC · endianness · expanding opcodes

QUICK TRICK

Instruction = **opcode + operand fields**. N instructions need $\lceil \log_2 N \rceil$ opcode bits. M addressable locations need $\log_2 M$ address bits. Expanding opcode: **(unused codes)** $\times 2^{\text{field bits}}$ gives the codes for the next level.

4.1 What is an instruction set?

● LEVEL 1 · BEGINNER

The **instruction set architecture (ISA)** is the list of operations the hardware understands and how they are encoded. It is the contract between software and hardware. **Analogy.** A restaurant menu: the menu (ISA) stays the same even if the kitchen (hardware) is rebuilt.

Category	Examples
Data transfer	LOAD, STORE, MOV, PUSH, POP
Arithmetic / logic	ADD, SUB, MUL, AND, OR, SHIFT, COMPARE
Control transfer	JUMP, BRANCH, CALL, RETURN
I/O and system	IN, OUT, HALT, interrupt return

Key registers: **PC** (address of the next instruction), **IR** (current instruction), **MAR/MDR** (memory address/data), **SP** (stack pointer), flags (Z, N, C, V).

4.2 Number of addresses

● LEVEL 2 · EXAM

Task: X = (A + B) × (C + D). Count instructions in each machine type.

Type	Program	Count
3-address	ADD R1,A,B · ADD R2,C,D · MUL X,R1,R2	**3**
2-address	MOV R1,A · ADD R1,B · MOV R2,C · ADD R2,D · MUL R1,R2 · MOV X,R1	**6**
1-address (accumulator)	LOAD A · ADD B · STORE T · LOAD C · ADD D · MUL T · STORE X	**7**
0-address (stack)	PUSH A · PUSH B · ADD · PUSH C · PUSH D · ADD · MUL · POP X	**8**

QUICK TRICK

More addresses → shorter programs but longer instructions. A stack machine uses the **postfix** form: (A + B) × C becomes A B + C ×.

4.3 Instruction format sizing

● LEVEL 2 · EXAM

- Instruction length = opcode + mode bits + register fields + address / immediate fields.
- **Example.** 32-bit instruction, 6-bit opcode, three register fields for 32 registers (3 × 5 = 15 bits) → 32 − 6 − 15 = **11 bits** left over.
- Addressable memory = $2^{(\text{address field bits})}$ locations. A 10-bit address field reaches 1024 words.
- **PC-relative branch:** an 8-bit signed offset reaches −128 … +127 words from the PC.

4.4 Expanding opcode

● LEVEL 3 · GATE

A fixed-length instruction can hold many 0- and 1-address instructions by reusing unused opcode patterns as **prefixes** for the next level.

Example. 16-bit instruction, each address field 4 bits. 3-address: opcode 4 bits (16 codes). Suppose 12 are used.

```
3-address : 16 codes, 12 used -> 4 left
2-address : 4 left x 2^4 = 64 codes (8-bit opcode), say 30 used -> 34 left
1-address : 34 left x 2^4 = 544 codes (12-bit opcode)  -> max 544 one-address instructions
0-address : (whatever is left) x 2^4 ...
```

4.5 RISC vs CISC, endianness

● LEVEL 2 · EXAM

	RISC	CISC
Instruction length	Fixed (e.g. 32 bit)	Variable
Addressing modes	Few	Many
Memory access	Only LOAD/STORE	Most instructions
Registers	Many (e.g. 32)	Fewer
Control unit	Hardwired, easy to pipeline	Often microprogrammed
CPI	Close to 1 (pipelined)	Higher, varies

Endianness. Store 0x12345678 at address 100. **Big-endian** (most significant byte first): 100: 12, 101: 34, 102: 56, 103: 78. **Little-endian** (least significant first): 100: 78, 101: 56, 102: 34, 103: 12.

4.6 Common mistakes

COMMON MISTAKE

- Counting address bits for a **word**-addressable memory as if it were byte-addressable.
- Forgetting that the leftover opcode codes (not the used ones) are expanded.
- Using $\lfloor \log_2 \rfloor$ instead of $\lceil \log_2 \rceil$ for opcode bits.
- Mixing up the number of instructions needed in 1-address vs 2-address code.

GATE QUICK CHECK — Chapter 4

```
opcode bits = ceil(log2 #instr)     address bits = log2 (#locations)
(A+B)*(C+D): 3-addr 3 | 2-addr 6 | 1-addr 7 | 0-addr 8
expanding opcode: next level codes = (codes left) x 2^(field bits)
big-endian: MSB at lowest address   RISC: fixed length, load/store only
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Minimum opcode bits for 60 instructions.

Q2. Address bits for a byte-addressable 1 MB memory.

Q3. Which uses fixed-length instructions, RISC or CISC?

Q4. Write A = (B + C) × D in 1-address code and count the instructions.

● LEVEL 2 · EXAM

Q5. 32-bit instruction, 6-bit opcode, three register fields for 32 registers. Bits left over?

Q6. 0x12345678 is stored at address 100. What byte is at address 100 in little-endian?

Q7. Postfix form and stack code for (A + B) × C.

Q8. 16-bit instruction with a 6-bit opcode and one 10-bit address. How many words can it address directly?

● LEVEL 3 · GATE

Q9. 16-bit instruction, 4-bit address fields. 12 three-address and 30 two-address instructions exist. Maximum one-address instructions?

Q10. A PC-relative branch uses an 8-bit signed offset (in words). Range of reachable targets?

Q11. How many different 2-address instructions can be added if all 15 of 16 three-address codes are used (4-bit fields, 16-bit instruction)?

Answers

Q	Solution
1	$\lceil \log_2 60 \rceil$ = **6**.
2	2^{20} bytes → **20 bits**.
3	**RISC**.
4	LOAD B · ADD C · MUL D · STORE A = **4**.
5	32 − 6 − 15 = **11 bits**.
6	**0x78** (least significant byte first).
7	A B + C ×: PUSH A · PUSH B · ADD · PUSH C · MUL · POP X.
8	2^{10} = **1024**.
9	16 − 12 = 4 → 4 × 16 = 64 two-address codes; 64 − 30 = 34 → 34 × 16 = **544**.
10	**−128 … +127** words from the PC.
11	16 − 15 = 1 code left → 1 × 16 = **16** two-address codes.

Next: Chapter 5 — Addressing Modes.

AMJESH singhamjesh.github.io

CHAPTER 5

Addressing Modes

Immediate · direct · indirect · register · register indirect · indexed · base · PC-relative · auto-increment

QUICK TRICK

EA = effective address of the operand. Immediate: operand is *in* the instruction. Direct: EA = A. Indirect: EA = M[A]. Indexed: EA = A + R. PC-relative: **EA = PC (already incremented) + offset**. Count the memory references.

5.1 Why many modes?

● LEVEL 1 · BEGINNER

The problem. The same instruction must reach constants, variables, array elements, pointers and code. **The idea.** The instruction names *how* to find its operand. **Analogy.** Finding a book: "this one in my hand" (immediate), "shelf 5" (direct), "the slip says which shelf" (indirect).

5.2 The modes

● LEVEL 2 · EXAM

Mode	Effective address / operand	Operand memory refs	Typical use
Implied	operand fixed (e.g. accumulator, stack top)	0	CLC, PUSH
Immediate	operand = value in instruction	0	Constants
Register	operand = R	0	Fast variables
Direct (absolute)	EA = A	1	Static variables
Indirect	EA = M[A]	2	Pointers
Register indirect	EA = (R)	1	Pointers, arrays
Indexed	EA = A + (index R)	1	Array elements
Base register	EA = (base R) + displacement	1	Relocation, records
PC-relative	EA = PC + offset	1 (for data)	Branches, position-independent code
Auto-increment	EA = (R); then R ← R + d	1	Walking through arrays
Auto-decrement	R ← R − d; then EA = (R)	1	Stacks

5.3 Worked example

● LEVEL 2 · EXAM

Memory: M[100] = 200, M[200] = 300. Registers: R1 = 100. Instruction field A = 100. Value loaded:

Mode	Value fetched	Reasoning
Immediate #100	**100**	the constant itself
Direct 100	**200**	M[100]
Indirect (100)	**300**	M[M[100]] = M[200]
Register R1	**100**	contents of R1
Register indirect (R1)	**200**	M[R1] = M[100]
Indexed 100(R1)	**300**	EA = 100 + 100 = 200, M[200]

5.4 PC-relative branches

● LEVEL 3 · GATE

When the instruction is executed the PC has **already been incremented** to the next instruction. **Example.** Branch at address 3000, instruction length 4 bytes, offset +12: target = 3000 + 4 + 12 = **3016**.

- **Auto-increment:** MOV (R1)+, R2 with R1 = 1000 and 4-byte words loads M[1000] into R2, then R1 becomes **1004**.
- **Total memory references** of an instruction = 1 (fetch, per word) + operand references. ADD R1, @100 (one-word instruction, indirect) = 1 + 2 = **3**.
- Base register mode lets a program be **relocated** by changing only the base register. Indexed mode keeps the array start in the instruction and the index in a register.

5.5 Common mistakes

COMMON MISTAKE

- Using the old PC (instruction address) instead of the incremented PC for PC-relative.
- Forgetting the extra memory access in indirect mode.
- Confusing indexed (address field + register) with register indirect (register only).
- Counting the instruction fetch as zero memory references.

QUICK TRICK

Memory line: "Immediate: here. Direct: there. Indirect: ask there where."

GATE QUICK CHECK — Chapter 5

```
Imm 0 | Reg 0 | Direct 1 | Reg-indirect 1 | Indexed 1 | Indirect 2   (operand memory refs)
Indexed EA=A+R   Base EA=BR+disp   PC-rel EA=PC_next+offset   Auto-inc EA=(R), R+=d
arrays -> indexed   pointers -> indirect   relocation -> base   branches -> PC-relative
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Which modes need no memory access for the operand?

Q2. How many memory references fetch an operand in indirect mode?

Q3. Which mode suits array access? Which suits position-independent branches?

Q4. Indexed mode: A = 500, index register = 20. EA?

● LEVEL 2 · EXAM

Q5. Using the example in 5.3, what does Indirect (100) fetch?

Q6. Branch at 3000, 4-byte instruction, offset +12 (PC-relative). Target?

Q7. MOV (R1)+, R2 with R1 = 1000, 4-byte words. Final R1?

Q8. Total memory references for ADD R1, @100 as a one-word instruction?

● LEVEL 3 · GATE

Q9. A 16-bit instruction uses base register mode with an 8-bit unsigned displacement. Range of EA with base register = 4000?

Q10. Why is PC-relative addressing good for position-independent code?

Q11. Instruction at 2000 occupies 2 bytes; PC-relative offset is −6. Target?

Answers

Q	Solution
1	**Immediate** and **register** (and implied).
2	**2**.
3	**Indexed**; **PC-relative**.
4	**520**.
5	**300**.
6	3000 + 4 + 12 = **3016**.
7	**1004**.
8	1 fetch + 2 (indirect) = **3**.
9	EA from **4000 to 4255**.
10	The target is expressed as a distance from the current instruction, so it stays correct wherever the program is loaded.
11	2000 + 2 − 6 = **1996**.

Next: Chapter 6 — ALU Design.

AMJESH
singhamjesh.github.io

CHAPTER 6

ALU Design

Full adder · ripple vs carry-lookahead · adder/subtractor · overflow flag · Booth's multiplication · division

QUICK TRICK

Full adder: $\mathbf{S = A \oplus B \oplus C_{in}}$, $\mathbf{C_{out} = AB + C_{in}(A \oplus B)}$. Carry lookahead: $\mathbf{G = AB}$, $\mathbf{P = A \oplus B}$, $\mathbf{C_{i+1} = G_i + P_iC_i}$. Subtract = add B' with $C_{in} = 1$. Booth: 10 → subtract M, 01 → add M, 00/11 → shift only.

6.1 What does an ALU do?

● LEVEL 1 · BEGINNER

The ALU performs arithmetic (add, subtract, multiply) and logic (AND, OR, XOR, NOT, shift). Its building block is the **1-bit adder**. Flags it sets: **Z** (zero), **N** (negative), **C** (carry), **V** (overflow).

6.2 Adders

● LEVEL 2 · EXAM

Circuit	Sum	Carry out
Half adder	$A \oplus B$	$A \cdot B$
Full adder	$A \oplus B \oplus C_{in}$	$AB + C_{in}(A \oplus B)$

- **Ripple-carry adder:** n full adders in a chain. The carry must travel through every stage, so delay grows **linearly with n**.
- **Carry-lookahead adder (CLA):** compute all carries in parallel from **generate** $\mathbf{G_i = A_iB_i}$ and **propagate** $\mathbf{P_i = A_i \oplus B_i}$:

```
C1 = G0 + P0 C0
C2 = G1 + P1 G0 + P1 P0 C0
C3 = G2 + P2 G1 + P2 P1 G0 + P2 P1 P0 C0          Si = Pi XOR Ci
Delay becomes (nearly) independent of n for a block; wider adders use groups of lookahead blocks.
```

Example. A = 1011 (11), B = 0110 (6), $C_0 = 0$. Bits LSB first: G = 0,1,0,0; P = 1,0,1,1. C1 = 0, C2 = 1, C3 = 1, C4 = 1. S = P ⊕ C = 1,0,0,0. Result = C4 S3 S2 S1 S0 = **10001 = 17** ✓.

6.3 Adder/subtractor and overflow

● LEVEL 2 · EXAM

```
B_i ---XOR---> to adder        M = 0 : A + B            M = 1 : A - B = A + B' + 1
M   ----^----> (also Cin)      n XOR gates, Cin = M
Overflow V = C_into_MSB XOR C_out_of_MSB
```

6.4 Multiplication

● LEVEL 3 · GATE

- **Shift-and-add (unsigned):** for each multiplier bit that is 1, add the shifted multiplicand. n-bit × n-bit gives a **2n-bit** product; n steps.
- **Array multiplier (n × n):** n^2 AND gates, $n(n-2)$ full adders, n half adders. (8 × 8: 64 AND, 48 FA, 8 HA.)

Booth's algorithm (signed multiplication)

Look at pairs (Q_0, Q_{-1}) of the multiplier (Q_{-1} starts at 0). **10:** A = A − M. **01:** A = A + M. **00 or 11:** do nothing. Then **arithmetic shift right** A, Q, Q_{-1}. Repeat n times. Result is in A:Q.

Example. M = 0011 (+3), Q = 1110 (−2), n = 4:

```
Step   A     Q     Q-1   Action
init   0000  1110  0
1      0000  0111  0     pair 00 -> shift
2      1110  1011  1     pair 10 -> A = A - M = 1101, shift
3      1111  0101  1     pair 11 -> shift
4      1111  1010  1     pair 11 -> shift
Product = A:Q = 1111 1010 = -6
```

QUICK TRICK

Booth's trick: a run of 1s is worth $(+2^{k+1} - 2^{j})$, so one subtraction at the run's start and one addition after its end. A multiplier like 00111100 needs only **2** add/subtract operations.

6.5 Division

● LEVEL 2 · EXAM

Restoring division: n steps. Shift the remainder:quotient left, subtract the divisor; if the result is negative restore it and set quotient bit 0, else keep it and set bit 1. **Non-restoring** avoids the restore step (it adds instead in the next step). Both take n iterations for n-bit operands.

6.6 Common mistakes

COMMON MISTAKE

- Saying a CLA removes the carry delay entirely (it reduces it; very wide adders use groups).
- Forgetting $C_{in} = 1$ when using the adder for subtraction.
- Using a 1-bit product width: n × n multiplication needs **2n** bits.
- Doing a logical shift instead of an **arithmetic** shift in Booth.

GATE QUICK CHECK — Chapter 6

```
FA: S=A^B^Cin  Cout=AB+Cin(A^B)      CLA: G=AB, P=A^B, C(i+1)=Gi+Pi*Ci, Si=Pi^Ci
A-B = A + B' + 1 (XOR gates + Cin=1)     V = C_in_MSB XOR C_out_MSB
Booth pair (Q0,Q-1): 10 sub | 01 add | 00,11 shift      product = 2n bits, n steps
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Sum and carry of a full adder for A = 1, B = 1, $C_{in} = 1$.

Q2. Why is a ripple-carry adder slow?

Q3. How many bits does the product of two 8-bit numbers need?

Q4. How does an adder subtract?

● LEVEL 2 · EXAM

Q5. 4-bit addition 0110 + 0101. Overflow flag?

Q6. Booth multiplication of (+3) × (−2) in 4 bits. Result?

Q7. How many add/subtract operations does Booth perform for multiplier 00111100?

Q8. CLA: A = 1011, B = 0110, $C_0 = 0$. Give P, G and the sum.

● LEVEL 3 · GATE

Q9. Number of gates in an 8 × 8 array multiplier (AND, full adders, half adders).

Q10. Restoring division of 8-bit numbers: how many iterations?

Q11. How many XOR gates does an n-bit adder/subtractor need (excluding overflow logic)?

Answers

Q	Solution
1	S = 1, C_{out} = **1**.
2	The carry passes through every stage, so delay grows with the number of bits.
3	**16**.
4	A + B' + 1: XOR B with 1 and set C_{in} = 1.
5	Sum 1011. Carry into MSB 1, carry out 0, so **V = 1**.
6	**1111 1010 = −6** (trace in 6.4).
7	**2** (one subtract at the start of the run of 1s, one add after its end).
8	P = 1101, G = 0010 (reading bit 3 to bit 0 gives P: 1,1,0,1 and G: 0,0,1,0). Sum = **10001** (17).
9	**64 AND, 48 full adders, 8 half adders**.
10	**8**.
11	**n** (one per bit of B).

Next: Chapter 7 — Control Unit: Hardwired and Microprogrammed.

AMJESH
singhamjesh.github.io

CHAPTER 7

Control Unit: Hardwired & Microprogrammed

Micro-operations · hardwired vs microprogrammed · horizontal vs vertical · control store sizing · microprogram sequencing

QUICK TRICK

Hardwired = **fast, rigid** (logic gates). Microprogrammed = **flexible, slower** (control store ROM). Horizontal microinstruction: **1 bit per control signal**, wide, fast. Vertical: **encoded fields**, narrow, needs decoders. A field of n mutually exclusive signals needs $\lceil \log_2(n+1) \rceil$ bits if "no-op" is allowed.

7.1 What does the control unit do?

● **LEVEL 1 · BEGINNER**

The control unit turns each instruction into a **sequence of control signals** (open this register, select that ALU operation, read memory) at the right time. **Analogy.** A conductor: the instrument (data path) plays, and the conductor decides who plays when.

Instruction fetch as micro-operations:

```
T0 : MAR <- PC
T1 : MDR <- M[MAR] ;  PC <- PC + 1
T2 : IR  <- MDR
T3 : decode, then execute micro-operations specific to the opcode
```

7.2 Hardwired control

● **LEVEL 2 · EXAM**

Control signals are produced by a fixed network of gates, decoders and a step counter from (opcode, flags, timing step). Built with state tables and logic minimisation.

7.3 Microprogrammed control

● **LEVEL 2 · EXAM**

Each instruction is implemented by a **microprogram** (a list of microinstructions) stored in a **control store** (ROM). A **microprogram counter (µPC / CAR)** holds the address of the next microinstruction. The opcode is mapped to the starting address of its routine.

	Hardwired	Microprogrammed
Speed	Faster	Slower (control-store access)
Flexibility	Hard to change	Easy: edit the microcode
Design effort	Complex for big ISAs	Systematic, simpler for CISC
Cost	More logic	Needs control memory
Typical	RISC	CISC

7.4 Microinstruction formats

● LEVEL 2 · EXAM

	Horizontal	Vertical
Encoding	One bit per control signal	Signals grouped into encoded fields
Width	Wide	Narrow
Parallelism	High: many signals in one step	Limited
Speed	Faster (no decoding)	Slower (decoders)
Control store	Larger (wide words)	Smaller

Word width = control field + next-address field (+ condition-select bits if given). **Horizontal example:** 40 control signals + 10-bit next address = **50 bits**. **Vertical example:** groups of 5, 8, 3 and 12 mutually exclusive signals, each with a no-op: $\lceil \log_2 6 \rceil + \lceil \log_2 9 \rceil + \lceil \log_2 4 \rceil + \lceil \log_2 13 \rceil = 3 + 4 + 2 + 4 =$ **13 bits**.

Control store size = (number of microinstructions) × (word width). Address bits = $\lceil \log_2(\text{number of words}) \rceil$. **Example:** 512 words × 32 bits = 16 Kbit = **2 KB**, 9 address bits.

7.5 Sequencing and mapping

● LEVEL 3 · GATE

- **Next-address field:** each microinstruction names the next one (or μPC just increments). Conditional branches use a condition-select field and a branch address.
- **Opcode to microaddress:** if each routine has at most 2^m microinstructions and the opcode is k bits, start address = (opcode followed by m zeros). Example: 6-bit opcode, ≤ 4 microinstructions each → start = opcode × 4, address width **8 bits**.
- **Nanoprogramming** (two-level control store) saves space when many microinstructions repeat.

7.6 Common mistakes

COMMON MISTAKE

- Saying vertical microcode is faster (horizontal is faster; vertical is smaller).
- Forgetting the next-address field in the control word width.
- Using $\lceil \log_2 n \rceil$ instead of $\lceil \log_2(n + 1) \rceil$ when a no-operation code is required.
- Forgetting the common fetch routine when counting total control-store words.

QUICK TRICK

Memory line: "Horizontal = wide and quick. Vertical = narrow and encoded."

GATE QUICK CHECK — Chapter 7

```
hardwired fast/rigid/RISC | microprogrammed flexible/slower/CISC
horizontal: width = #signals (+next addr)    vertical: sum ceil(log2(n_i+1))
control store bits = words x width           address bits = ceil(log2 words)
fetch: MAR<-PC ; MDR<-M[MAR], PC<-PC+1 ; IR<-MDR
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Which control unit is faster, hardwired or microprogrammed?

Q2. Which is easier to modify?

Q3. What does the micro-program counter hold?

Q4. Which format needs decoders, horizontal or vertical?

● LEVEL 2 · EXAM

Q5. Horizontal microinstruction: 40 control signals and a 10-bit next-address field. Word width?

Q6. Vertical format: four groups of 5, 8, 3 and 12 mutually exclusive signals (no-op allowed). Control bits?

Q7. Control store of 512 words × 32 bits: size in KB and address bits.

Q8. Micro-operations for the instruction fetch.

● **LEVEL 3 · GATE**

Q9. A machine has 64 opcodes, each routine averages 8 microinstructions, plus a 12-word common fetch routine. Minimum control-store address bits?

Q10. 6-bit opcode and every routine at most 4 microinstructions. How is the start address formed and how wide is it?

Q11. Why does horizontal microcode allow more parallelism?

Answers

Q	Solution
1	**Hardwired**.
2	**Microprogrammed** (change the microcode).
3	The **address of the next microinstruction**.
4	**Vertical**.
5	40 + 10 = **50 bits**.
6	3 + 4 + 2 + 4 = **13 bits**.
7	512 × 32 = 16384 bits = **2 KB**; **9 address bits**.
8	MAR ← PC; MDR ← M[MAR] and PC ← PC + 1; IR ← MDR.
9	64 × 8 + 12 = 524 words → $\lceil \log_2 524 \rceil$ = **10 bits**.
10	Start address = opcode followed by 2 zero bits (opcode × 4). Width = 6 + 2 = **8 bits**.
11	Each control signal has its own bit, so any combination of signals can be active in the same step.

Next: Chapter 8 — Instruction Pipelining and Hazards.

AMJESH singhamjesh.github.io

CHAPTER 8

Instruction Pipelining & Hazards

Stages · speedup · stall counting · structural, data, control hazards · forwarding · branch penalty

QUICK TRICK

k stages, n instructions: **cycles = k + n − 1**. Speedup = **n·k / (k + n − 1)** → k. Clock = **slowest stage**. Pipelined CPI = **1 + stall cycles per instruction**. RAW without forwarding: **2 stalls**; with forwarding: **0** (ALU→ALU), **1** (load-use).

8.1 The idea

● **LEVEL 1 · BEGINNER**

The problem. Executing one instruction fully before starting the next leaves most hardware idle. **The idea.** Overlap the instructions, like an assembly line: while one instruction is being executed, the next is being decoded and a third fetched.

Analogy. A laundry with washer, dryer and folder. Start the next load as soon as the washer is free, not after the whole first load is folded.

Classic 5-stage pipeline: **IF** (fetch) → **ID** (decode, read registers) → **EX** (ALU) → **MEM** (data memory) → **WB** (write back).

8.2 Time and speedup

● **LEVEL 2 · EXAM**

```
Cycle for pipelined clock   = max(stage delay) (+ latch delay)
Time for n instructions     = (k + n - 1) x clock
Non-pipelined time          = n x (sum of stage delays)
Speedup (equal stages)      = n*k / (k + n - 1)   ->  k as n grows
Efficiency = speedup / k      Throughput = n / ((k + n - 1) x clock)
```

Example. 5 stages with delays 10, 8, 12, 9, 6 ns. Non-pipelined: 45 ns per instruction. Pipelined clock = 12 ns. For 100 instructions: pipelined (5 + 99) × 12 = **1248 ns**; non-pipelined 100 × 45 = 4500 ns; speedup ≈ **3.6** (not 5, because the stages are unbalanced).

QUICK TRICK

Speedup can never exceed the number of stages k. Unbalanced stages and stalls both reduce it. With ideal CPI = 1: **speedup = k / (1 + stalls per instruction)** (same clock).

8.3 Hazards

● **LEVEL 2 · EXAM**

Hazard	Cause	Cure
Structural	Two instructions need the same hardware in one cycle (e.g. one memory for IF and MEM)	Duplicate resources (split I/D cache), or stall
Data	An instruction needs a result not yet written. **RAW** (true), WAR, WAW	Forwarding (bypass), stalls, compiler scheduling
Control	Branch target not known until later	Stall, predict, delayed branch

In the simple in-order 5-stage pipeline only **RAW** hazards happen. WAR and WAW appear with out-of-order execution.

Counting RAW stalls (5-stage, register file written in the first half-cycle and read in the second)

```
I1: ADD R1,R2,R3       I2: SUB R4,R1,R5   (I2 needs R1)
No forwarding: I1 writes R1 in WB (cycle 5); I2 can read it in cycle 5.
cycle:  1  2  3  4  5  6  7  8
I1      IF ID EX ME WB
I2         IF ID ID ID EX ME WB       -> 2 stall cycles
With forwarding (EX result of I1 -> EX of I2): 0 stalls.
```

Producer → consumer (adjacent)	No forwarding	With forwarding
ALU → ALU	2 stalls	**0**
Load → use (load-use)	2 stalls	**1** (data exists only after MEM)
Two apart	1 stall	0

Three-instruction example. I1: LW R1,0(R2); I2: ADD R3,R1,R4; I3: SUB R5,R3,R6. **With forwarding:** I2 stalls 1 (load-use); I3 needs no stall. Total = (5 + 3 − 1) + 1 = **8 cycles**. **Without forwarding:** I2 stalls 2, I3 stalls 2 → 7 + 4 = **11 cycles**.

8.4 Control hazards

● LEVEL 2 · EXAM

- Branch penalty = cycles lost until the branch outcome and target are known (resolved in EX: 2 cycles; in ID: 1 cycle).
- **Policies:** stall always · predict not-taken (penalty only when taken) · predict taken · **delayed branch** (fill the slot after the branch with a useful instruction) · dynamic prediction (1-bit, 2-bit counters).
- **CPI = 1 + (branch frequency) × (fraction of branches paying the penalty) × (penalty)**. Example: 20% branches, 60% taken, predict not-taken, penalty 2: CPI = 1 + 0.2 × 0.6 × 2 = **1.24**.
- **Structural example:** single memory port, 30% loads/stores each stall instruction fetch by 1 cycle: CPI = 1 + 0.3 = **1.3**.

8.5 Common mistakes

COMMON MISTAKE

- Using n × k cycles instead of k + n − 1 for a pipeline.
- Dividing the non-pipelined time by the **sum** of stages for the clock (the clock is the **max** stage).
- Forgetting the 1-cycle load-use stall even with forwarding.
- Counting WAR/WAW hazards in a simple in-order pipeline.
- Forgetting latch/register delay when the question gives it (add it to every stage).

QUICK TRICK

Memory line: "First instruction pays k cycles; every later one adds just one."

GATE QUICK CHECK — Chapter 8

```
cycles = k+n-1      speedup = nk/(k+n-1) <= k      clock = max stage (+latch)
CPI = 1 + stalls/instr      speedup = k/(1+stalls)
RAW adjacent: no fwd 2 stalls | fwd ALU 0 | fwd load-use 1
branch CPI = 1 + f_branch * f_penalised * penalty
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Cycles to execute 10 instructions in a 5-stage pipeline (no stalls).

Q2. Maximum possible speedup of a k-stage pipeline.

Q3. Which hazards can occur in a simple in-order 5-stage pipeline?

Q4. RAW stalls between adjacent ALU instructions with forwarding.

● LEVEL 2 · EXAM

Q5. Speedup for n = 100 instructions on a 5-stage pipeline with equal stages. Efficiency?

Q6. Stage delays 10, 8, 12, 9, 6 ns. Time for 100 instructions pipelined and speedup over non-pipelined?

Q7. Adjacent RAW dependence, no forwarding, register file split-cycle. Stall cycles?

Q8. 20% branches, 60% of them taken, predict not-taken with penalty 2. CPI?

● LEVEL 3 · GATE

Q9. LW R1,0(R2); ADD R3,R1,R4; SUB R5,R3,R6 with forwarding. Total cycles?

Q10. Same code without forwarding (split-cycle register file). Total cycles?

Q11. A single-ported memory causes 1 stall cycle for every load/store, which are 30% of instructions. CPI?

Q12. Stages need 200, 150, 300, 250, 100 ps and each pipeline register adds 20 ps. Clock cycle and speedup for a long program (vs non-pipelined 1000 ps)?

Answers

Q	Solution
1	5 + 10 − 1 = **14**.
2	**k**.
3	Structural, RAW data hazards and control hazards (WAR/WAW only with out-of-order execution).
4	**0**.
5	Speedup = 500/104 ≈ **4.81**; efficiency = 4.81/5 ≈ **96.2%**.
6	Clock 12 ns; (5 + 99) × 12 = **1248 ns**. Non-pipelined 100 × 45 = 4500 ns. Speedup ≈ **3.6**.
7	**2**.
8	1 + 0.2 × 0.6 × 2 = **1.24**.
9	I2 stalls 1 (load-use), I3 none: (5 + 3 − 1) + 1 = **8**.
10	I2: 2 stalls, I3: 2 stalls: 7 + 4 = **11**.
11	1 + 0.3 × 1 = **1.3**.
12	Clock = 300 + 20 = **320 ps**. Speedup for large n = 1000/320 ≈ **3.1** (non-pipelined has no register overhead).

Next: Chapter 9 — Memory Hierarchy, Performance and Interfacing.

CHAPTER 9

Memory Hierarchy, Performance & Interfacing

Locality · AMAT · hierarchical vs simultaneous access · chip count · address decoding · interleaving

QUICK TRICK

AMAT = hit time + miss rate × miss penalty. Two levels: $H_1 + m_1(H_2 + m_2 \cdot P_{mm})$. Chips needed = **(total bits) ÷ (chip bits)**. Address bits = $\log_2$**(number of locations)**.

9.1 The hierarchy

● LEVEL 1 · BEGINNER

The problem. Fast memory is small and costly; big memory is slow and cheap. **The idea.** Stack memories from small/fast to big/slow so that most accesses hit the fast level, because programs show **locality**. **Analogy.** Your desk (cache), your bookshelf (main memory), the library (disk).

Level	Speed	Size	Cost / bit
Registers	fastest	tiny	highest
Cache (SRAM)	ns	KB – MB	high
Main memory (DRAM)	~50–100 ns	GB	medium
Secondary storage	ms	TB	lowest

- **Temporal locality:** a recently used item is likely to be used again soon (loops).
- **Spatial locality:** items near a used item are likely to be used (arrays, sequential code). Blocks (lines) exploit this.
- **SRAM:** flip-flops, fast, no refresh, costly (cache). **DRAM:** capacitors, needs refresh, dense and cheap (main memory).

9.2 Performance of a hierarchy

● LEVEL 2 · EXAM

```
AMAT = Hit time + Miss rate x Miss penalty
2-level: AMAT = H1 + m1 x (H2 + m2 x Pmem)          (m2 = local miss rate of L2)
Effective access time (hit ratio h, cache tc, memory tm):
   Simultaneous (parallel) access : T = h*tc + (1-h)*tm
   Hierarchical (sequential)      : T = h*tc + (1-h)*(tc + tm)
```

Example. Hit time 2 ns, miss rate 5%, penalty 100 ns: AMAT = 2 + 0.05 × 100 = **7 ns**. **Two level:** H_1 = 1 ns, m_1 = 10%, H_2 = 10 ns, m_2 = 20%, memory 100 ns: AMAT = 1 + 0.1 × (10 + 0.2 × 100) = **4 ns**.

Example. h = 0.9, t_c = 10 ns, t_m = 100 ns. Simultaneous: 9 + 10 = **19 ns**. Hierarchical: 9 + 0.1 × 110 = **20 ns**. **Speedup due to cache** = t_m / T = 100/19 ≈ 5.3.

COMMON MISTAKE

- Always read whether the question uses simultaneous or hierarchical access.

9.3 Memory interfacing (building memory from chips)

● LEVEL 2 · EXAM

- Memory size = (number of locations) × (word width). Chip = 2^a locations × d bits.
- **Chips needed** = (required locations ÷ chip locations) × (required width ÷ chip width).
- Arrange as **rows × columns**: columns widen the word, rows add locations. The high-order address bits go to a **decoder** that picks the row (chip select); the low-order bits go to every chip.

Example. Build a 64K × 16 memory from 16K × 8 chips. Chips = (64K/16K) × (16/8) = 4 × 2 = **8**: 4 rows × 2 columns. Address bits = 16. A_{13}...A_0 go to all chips; $A_{15}A_{14}$ drive a **2-to-4 decoder** selecting the row.

```
A15 A14 -> [2-to-4 decoder] -> CS of row0..row3
A13..A0 -> every chip          each row has 2 chips side by side: D15..D8 | D7..D0
```

Example. 16K × 8 memory from 4K × 8 chips: 4 chips, address 14 bits; $A_{13}A_{12}$ → 2-to-4 decoder; A_{11}...A_0 to each chip.

9.4 Address counting and interleaving

● LEVEL 2 · EXAM

- 64 KB byte-addressable → 16 address lines. If the machine is 32-bit word-addressable with 64 KB of storage: 16K words → 14 address lines.
- **Low-order interleaving:** with m banks ($m = 2^j$), the low j address bits choose the bank, so consecutive addresses fall in different banks and can be accessed in an overlapped way (good for sequential access).
- **High-order interleaving** = ordinary contiguous banks (good for adding capacity, no overlap).

9.5 Common mistakes

COMMON MISTAKE

- Mixing up bits and bytes in the chip-count formula.
- Using the global miss rate for L2 where the formula needs the **local** miss rate (or the other way round).
- Forgetting the 1 ns / 10 ns hit times when expanding a 2-level AMAT.
- Counting address lines for words when the question says byte-addressable.

GATE QUICK CHECK — Chapter 9

```
AMAT = Th + m*Tp       2-level: H1 + m1(H2 + m2*Pmm)
parallel: h*tc+(1-h)*tm     hierarchical: h*tc+(1-h)(tc+tm)
chips = (locs/chip_locs) x (width/chip_width)     address lines = log2(locs)
SRAM cache fast/costly | DRAM main memory refresh | low-order interleave -> consecutive banks
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Hit time 2 ns, miss rate 5%, miss penalty 100 ns. AMAT?

Q2. Address lines for a byte-addressable 64 KB memory.

Q3. Which memory is used for cache and which for main memory?

Q4. Name the two kinds of locality.

● LEVEL 2 · EXAM

Q5. h = 0.9, t_c = 10 ns, t_m = 100 ns. Effective time with simultaneous access and with hierarchical access?

Q6. Two-level cache: H_1 = 1 ns, m_1 = 10%, H_2 = 10 ns, local m_2 = 20%, memory 100 ns. AMAT?

Q7. How many 256K × 4 chips build a 1M × 8 memory? Arrangement?

Q8. Maximum miss rate so AMAT ≤ 3 ns with hit time 1 ns and penalty 50 ns?

● LEVEL 3 · GATE

Q9. Build 16K × 8 memory from 4K × 8 chips. Chips, address bits, and the decoder inputs?

Q10. Cache 20 ns, memory 200 ns, hit ratio 0.95 (simultaneous). Effective time and speedup due to the cache?

Q11. Low-order interleaving with 4 banks. Which bank holds address 13?

Answers

Q	Solution
1	2 + 0.05 × 100 = **7 ns**.
2	**16**.
3	Cache: **SRAM**. Main memory: **DRAM**.
4	**Temporal** and **spatial**.
5	Simultaneous: 9 + 10 = **19 ns**. Hierarchical: 9 + 0.1 × 110 = **20 ns**.
6	1 + 0.1 × (10 + 20) = **4 ns**.
7	(1M/256K) × (8/4) = 4 × 2 = **8 chips**: 4 rows × 2 columns.
8	1 + 50m ≤ 3 → m ≤ **0.04 (4%)**.
9	**4 chips**, 14 address bits; decoder: 2-to-4 on **A13, A12**; A11…A0 to every chip.
10	T = 19 + 10 = **29 ns**. Speedup = 200/29 ≈ **6.9**.
11	13 mod 4 = **bank 1**.

Next: Chapter 10 — Cache Memory Mapping.

AMJESH
singhamjesh.github.io

CHAPTER 10

Cache Memory Mapping

Direct · fully associative · set-associative · tag/index/offset · cache size in bits · replacement · write policies · miss types

QUICK TRICK

Address = **Tag | Index | Offset**. Offset = $\log_2$(block size). Direct: index = $\log_2$(lines). k-way: index = $\log_2$(lines / k). Fully associative: **no index**. Tag = address bits − index − offset. Cache bits = lines × (data + tag + valid [+ dirty]).

10.1 The problem and the idea

● **LEVEL 1 · BEGINNER**

The problem. The cache is much smaller than main memory. Which cache line may hold which memory block, and how do we find it? **The idea.** A **mapping function** decides. **Analogy.** A coat check: direct = your number fixes your hook; fully associative = any free hook; set-associative = any free hook in your row.

10.2 Three mappings

● **LEVEL 2 · EXAM**

Mapping	Where a block may go	Index	Comparators	Conflict misses
Direct-mapped	Exactly one line: (block number) mod (lines)	$\log_2$(lines)	1	Most
Fully associative	Any line	none	= number of lines	None
k-way set-associative	Any of k lines in set (block mod sets)	$\log_2$(lines / k)	k	In between

Direct-mapped is k = 1; fully associative is k = number of lines.

Worked example

32-bit address, 64 KB cache, 32 B blocks. Offset = 5 bits. Lines = $2^{16}/2^5 = 2^{11} = 2048$.

Mapping	Sets	Index	Tag
Direct-mapped	2048	11	32 − 11 − 5 = **16**
4-way	512	9	32 − 9 − 5 = **18**
Fully associative	1	0	32 − 5 = **27**

Total cache size (direct-mapped). Each line = 256 data bits + 16 tag + 1 valid = 273 bits. Total = 2048 × 273 = **559,104 bits ≈ 68.25 KB**. With a dirty bit (write-back): 2048 × 274 = 561,152 bits.

10.3 Tracing hits and misses

● **LEVEL 3 · GATE**

4 cache lines, 1-word blocks. Block addresses: **0, 4, 0, 8, 4, 12**. Compare the three mappings (LRU for associative).

Mapping	Where all of them go	Misses	Hits
Direct (4 lines)	all map to line 0 (address mod 4 = 0)	**6**	0
2-way (2 sets, set 0)	all in set 0 (mod 2 = 0); LRU evicts	**5**	1 (the second 0)
Fully associative (4 lines)	any line; only 4 distinct blocks	**4**	2 (0 and 4)

QUICK TRICK

More associativity → fewer conflict misses, but more comparators, more hardware and a longer hit time.

10.4 Replacement and write policies

● LEVEL 2 · EXAM

- **Replacement** (needed for associative caches): **LRU** (least recently used), **FIFO**, **random**. Direct-mapped needs none.
- **Write-through:** write to cache and memory together (simple, slow; use a write buffer). **Write-back:** write only the cache and set a **dirty bit**; write the line to memory when it is evicted.
- **Write miss: write-allocate** (load the block, then write; usual with write-back) or **no-write-allocate** (write straight to memory; usual with write-through).

10.5 Miss types and performance

● LEVEL 2 · EXAM

Miss	Cause	Reduced by
Compulsory (cold)	First access to a block	Larger blocks, prefetch
Capacity	Cache too small for the working set	Larger cache
Conflict	Too many blocks map to one set	More associativity

```
CPI = CPI_base + (memory accesses per instruction) x miss rate x miss penalty (cycles)
Example: 1.2 + 1.3 x 0.04 x 50 = 1.2 + 2.6 = 3.8
```

Block size effect. Larger blocks use spatial locality (fewer compulsory misses) but raise the miss penalty and, for a fixed cache, give fewer lines (more conflict misses).

10.6 Common mistakes

COMMON MISTAKE

- Taking the index as $\log_2$(lines) for a set-associative cache (use sets).
- Forgetting the valid bit (and dirty bit) when asked for the **total** cache size.
- Counting the tag from the wrong address width (use the physical address if the cache is physically addressed).
- Forgetting that fully associative has no index field.
- Using the byte address where the block address is needed in a hit/miss trace.

GATE QUICK CHECK — Chapter 10

```
offset = log2(block)   lines = cache/block   sets = lines/k   index = log2(sets)
tag = A - index - offset      direct: largest index, smallest tag | fully assoc: no index, largest tag
cache bits = lines x (8*block + tag + valid [+ dirty])      comparators = k
write-back: dirty bit, write on evict | write-through: always write memory
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Number of comparators in an 8-way set-associative cache.

Q2. Which miss type does full associativity eliminate?

Q3. Does a direct-mapped cache need a replacement policy?

Q4. What does a dirty bit mean?

● LEVEL 2 · EXAM

Q5. 32-bit address, 16 KB direct-mapped cache, 16 B blocks. Offset, index, tag bits?

Q6. Same cache but 4-way set-associative. Index and tag bits?

Q7. 64 KB direct-mapped cache, 32 B blocks, 32-bit address, valid bit only. Total bits?

Q8. Base CPI 1.2, 1.3 memory accesses per instruction, miss rate 4%, miss penalty 50 cycles. CPI?

● LEVEL 3 · GATE

Q9. Physical address 40 bits, 32 KB 4-way cache, 64 B blocks. Offset, index, tag?

Q10. Main memory 16 MB (24-bit address), 128 KB direct-mapped cache, 64 B blocks. Tag, index, offset?

Q11. Block addresses 0, 4, 0, 8, 4, 12 on a 4-line direct-mapped cache. Misses?

Q12. A 2-way cache with 2 sets, LRU, same sequence 0, 4, 0, 8, 4, 12. Misses?

Answers

Q	Solution
1	**8**.
2	**Conflict** misses.
3	**No**: each block has only one possible line.
4	The cache line has been modified and must be written back to memory on eviction (write-back cache).
5	Offset 4; lines 1024 → index 10; tag = 32 − 14 = **18**.
6	Sets = 256 → index **8**; tag = 32 − 8 − 4 = **20**.
7	2048 × (256 + 16 + 1) = **559,104 bits** (68.25 KB).
8	1.2 + 1.3 × 0.04 × 50 = **3.8**.
9	Offset 6. Lines = 512; sets = 128 → index **7**. Tag = 40 − 7 − 6 = **27**.
10	Offset 6; lines = 2048 → index 11; tag = 24 − 17 = **7**.
11	All map to line 0 → **6 misses**.
12	**5 misses**, 1 hit.

Next: Chapter 11 — Disk and Secondary Storage.

AMJESH singhamjesh.github.io

CHAPTER 11

Disk & Secondary Storage

Disk geometry · capacity · access time = seek + rotation + transfer · transfer rate · IOPS

QUICK TRICK

Access time = seek + rotational latency + transfer time (+ controller overhead). Average rotational latency = **half a rotation = 30 / RPM seconds**. Capacity = surfaces × tracks × sectors × bytes per sector.

(The 2027 syllabus wording does not name secondary storage. These are short, easy marks if asked, so keep this page for revision.)

11.1 Disk structure

● **LEVEL 1 · BEGINNER**

A hard disk has one or more **platters** spinning together. Each platter surface has concentric **tracks**; each track is split into **sectors** (the smallest unit read/written, often 512 B or 4 KB). The same track number on all surfaces forms a **cylinder**. A head per surface moves in and out together.

Analogy. A record player: moving the needle to the right groove (seek), waiting for the right spot to come around (rotation), then playing it (transfer).

11.2 Capacity

● **LEVEL 2 · EXAM**

```
Capacity = (surfaces) x (tracks per surface) x (sectors per track) x (bytes per sector)
Cylinders = tracks per surface        Surfaces = 2 x platters (if both sides used)
```

Example. 8 surfaces, 1000 tracks, 200 sectors per track, 512 B per sector: 8 × 1000 × 200 × 512 = **819,200,000 B** (about 819 MB, 781 MiB).

11.3 Access time

● **LEVEL 2 · EXAM**

Component	Meaning	Formula
Seek time	Move head to the correct track	given (often "average seek")
Rotational latency	Wait for the sector to arrive under the head	average = ½ × (60 / RPM)
Transfer time	Read the data	bytes ÷ transfer rate, or (sectors ÷ sectors per track) × rotation time
Controller overhead	Electronics	given

- **Rotation time** = 60 / RPM seconds. 7200 RPM → 8.33 ms; average latency = **4.17 ms**. 15,000 RPM → 4 ms; latency = **2 ms**.
- **Transfer rate** = (sectors per track × bytes per sector) ÷ rotation time. 200 sectors × 512 B at 6000 RPM (10 ms per rotation) → 102,400 B / 10 ms = **10.24 MB/s**.

Example. Average seek 6 ms, 7200 RPM, 100 sectors per track, read one sector: latency 4.17 ms, transfer = 8.33 ms / 100 = 0.083 ms. Access time = 6 + 4.167 + 0.083 = **10.25 ms**.

Example. 4 KB block (8 sectors of 512 B) on a track of 100 sectors at 7200 RPM, seek 9 ms: transfer = 8/100 × 8.33 = 0.67 ms. Total = 9 + 4.17 + 0.67 = **13.84 ms**, so about 1/0.01384 ≈ **72 random I/Os per second**.

QUICK TRICK

Random access is dominated by **seek + rotation**; sequential access mostly pays the transfer time. That is why contiguous layout is fast.

11.4 HDD vs SSD

● LEVEL 1 · BEGINNER

	HDD	SSD
Mechanism	Spinning platters, moving head	Flash memory, no moving parts
Random access	Slow (ms)	Fast (tens of µs)
Cost per GB	Lower	Higher
Wear	Mechanical	Limited write cycles

11.5 Common mistakes

COMMON MISTAKE

- Using a full rotation instead of half for the **average** rotational latency.
- Forgetting that transfer time depends on the fraction of the track read.
- Forgetting to double the platter count to get surfaces.
- Mixing ms and µs.

GATE QUICK CHECK — Chapter 11

```
access = seek + 0.5*(60/RPM) + transfer (+overhead)        rotation = 60/RPM
transfer rate = (sectors*bytes)/rotation time              capacity = surf*tracks*sect*bytes
7200 RPM -> latency 4.17 ms | 15000 RPM -> 2 ms
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Average rotational latency at 7200 RPM.

Q2. Average rotational latency at 15,000 RPM.

Q3. What is a cylinder?

Q4. Which part dominates random access time on an HDD?

● LEVEL 2 · EXAM

Q5. Capacity: 8 surfaces, 1000 tracks, 200 sectors per track, 512 B per sector.

Q6. 200 sectors of 512 B per track, 6000 RPM. Transfer rate?

Q7. Seek 6 ms, 7200 RPM, 100 sectors per track. Time to read one sector?

Q8. How many surfaces does a 2-platter disk (both sides used) have?

● LEVEL 3 · GATE

Q9. 4 KB block (8 sectors) on a 100-sector track, 7200 RPM, seek 9 ms. Access time and random IOPS?

Q10. 2 platters (4 surfaces), 10,000 tracks per surface, 500 sectors per track, 512 B sectors. Capacity?

Q11. Why is reading a large file stored contiguously faster than the same file scattered over the disk?

Answers

Q	Solution
1	0.5 × 8.33 = **4.17 ms**.
2	0.5 × 4 ms = **2 ms**.
3	The set of tracks with the same track number on all surfaces.
4	**Seek and rotational latency**.
5	8 × 1000 × 200 × 512 = **819,200,000 B**.
6	102,400 B per 10 ms = **10.24 MB/s**.
7	6 + 4.167 + 0.083 = **10.25 ms**.
8	**4**.
9	Transfer 0.67 ms. Total 9 + 4.17 + 0.67 = **13.84 ms** → ≈ **72 IOPS**.
10	4 × 10,000 × 500 × 512 = **10.24×10^9 B** (10.24 GB).
11	Contiguous data needs one seek and one rotational wait, then streams at the transfer rate; scattered data pays seek and rotation for every piece.

Next: Chapter 12 — I/O: Interrupt and DMA.

AMJESH
singhamjesh.github.io

CHAPTER 12

I/O: Interrupt & DMA

I/O interface · memory-mapped vs isolated · programmed I/O · interrupt-driven · DMA (burst, cycle stealing) · CPU overhead calculations

QUICK TRICK

Three ways: **programmed I/O** (CPU polls, wastes time), **interrupt-driven** (device interrupts the CPU per transfer), **DMA** (controller moves a whole block, one interrupt at the end). Overhead = **(events per second × cycles per event) ÷ clock rate**.

12.1 I/O interface

● **LEVEL 1 · BEGINNER**

An **I/O controller (interface)** sits between the CPU and a device. It has **status**, **command/control** and **data** registers. **Analogy.** A receptionist who takes messages for a busy manager.

	Memory-mapped I/O	Isolated (port-mapped) I/O
Address space	Shared with memory	Separate I/O space
Instructions	Ordinary LOAD / STORE	Special IN / OUT
Control signal	Same memory read/write lines	Separate I/O read/write line
Drawback	Uses up some memory addresses	Needs extra instructions and a control line

12.2 Programmed I/O and interrupts

● **LEVEL 2 · EXAM**

- **Programmed I/O (polling):** the CPU loops reading the status register until the device is ready. Simple, but the CPU is **busy-waiting**.
- **Interrupt-driven I/O:** the CPU does other work; the device raises an **interrupt request (IRQ)** when ready. Efficient for slow, sporadic devices.

```
Interrupt sequence
1. Device raises IRQ              2. CPU finishes the current instruction
3. CPU acknowledges, identifies the source (vector number)
4. CPU saves PC and flags (state)  5. Jumps to the Interrupt Service Routine (ISR)
6. ISR runs; returns with IRET (restores PC and flags)
```

- **Vectored** interrupt: the device supplies the ISR address (vector). **Non-vectored:** a fixed ISR polls devices to find the source.
- **Priority: daisy chain** (the device nearest the CPU wins), or a priority encoder. **Maskable** interrupts can be disabled; **non-maskable (NMI)** cannot. Nested interrupts allow a higher priority to pre-empt an ISR.
- **Exceptions/traps** (divide by zero, system call) are internal (software) interrupts. Number of vectors with an 8-bit vector number = 2^8 = **256**.

12.3 DMA

● **LEVEL 2 · EXAM**

The problem. Moving a large block word by word through the CPU wastes it. **The idea.** A **DMA controller** moves data between the device and memory directly. It has an **address register**, a **word count register** and a **control register**.

```
1. CPU programs the DMA controller: memory address, count, direction  (then goes on working)
2. Device is ready -> DMA asks for the bus (bus request / HOLD)
3. CPU grants it (bus grant / HLDA)        4. DMA transfers words directly to/from memory
5. After the last word DMA raises ONE interrupt: transfer complete
```

DMA mode	How the bus is used	CPU effect
Burst (block)	DMA holds the bus for the **whole block**	CPU locked out for the entire block
Cycle stealing	DMA takes the bus for **one memory cycle** per word	CPU slowed slightly each time
Transparent	DMA uses only cycles when the CPU is not using the bus	No slowdown, complex hardware

12.4 Overhead calculations

● LEVEL 3 · GATE

Polling a mouse: 30 polls/s × 400 cycles on a 500 MHz CPU = $12{,}000/5\times10^8$ = **0.0024%** of the CPU.

Polling a disk delivering 4 MB/s in 16 B chunks: $4\times10^6/16$ = 250,000 polls/s × 400 cycles = 10^8 cycles/s = **20%** of 500 MHz.

DMA for the same disk, 8 KB transfers, setup 1000 cycles + completion interrupt 500 cycles: transfers/s = $4\times10^6/8192 \approx 488$; overhead = 488 × 1500 = 732,000 cycles/s ≈ **0.146%**.

Cycle stealing: device 800 KB/s, 4-byte words → 200,000 words/s; each steals a 100 ns memory cycle: 200,000 × 100 ns = 20 ms per second = **2%** of the CPU time. **Burst:** a block of 1000 words with a 100 ns cycle blocks the CPU for **100 µs**.

12.5 Common mistakes

COMMON MISTAKE

- Saying DMA needs one interrupt per word (it needs **one** per block).
- Forgetting that the CPU still sets up the DMA controller.
- Mixing bytes per second and words per second in the overhead formula.
- Treating cycle stealing and burst mode as the same.

QUICK TRICK

Memory line: "Polling asks, interrupt tells, DMA does the carrying."

GATE QUICK CHECK — Chapter 12

```
overhead = (events/s x cycles/event) / f_clk
polling: busy-wait | interrupt: per transfer | DMA: setup + 1 interrupt per block
burst: CPU blocked for whole block | cycle stealing: 1 memory cycle per word
memory-mapped: LOAD/STORE | isolated: IN/OUT | daisy chain: nearest device wins
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Which I/O method wastes CPU time busy-waiting?

Q2. How does the DMA controller tell the CPU a transfer has finished?

Q3. In a daisy-chain priority scheme which device has the highest priority?

Q4. How many interrupt vectors can an 8-bit vector number address?

● LEVEL 2 · EXAM

Q5. A mouse is polled 30 times per second at 400 cycles per poll on a 500 MHz CPU. CPU fraction used?

Q6. A disk delivers 4 MB/s in 16 B chunks and is polled for each chunk (400 cycles per poll, 500 MHz). CPU fraction?

Q7. Cycle-stealing DMA: 800 KB/s device, 4-byte words, 100 ns memory cycle. CPU time lost?

Q8. Burst DMA of 1000 words, 100 ns per word. How long is the CPU locked out?

● LEVEL 3 · GATE

Q9. DMA for the 4 MB/s disk: 8 KB transfers, 1000 cycles setup + 500 cycles completion interrupt, 500 MHz CPU. CPU fraction?

Q10. Why can memory-mapped I/O use ordinary LOAD/STORE instructions?

Q11. List the order of events when an interrupt is taken.

Answers

Q	Solution
1	**Programmed I/O (polling)**.
2	With an **interrupt**.
3	The one **closest to the CPU**.
4	**256**.
5	$30 \times 400 / 5 \times 10^8$ = **0.0024%**.
6	$250{,}000 \times 400 = 10^8$ cycles/s → **20%**.
7	200,000 × 100 ns = 20 ms/s = **2%**.
8	**100 µs**.
9	Transfers/s = $4 \times 10^6 / 8192 \approx 488$. Cycles = 488 × 1500 ≈ 732,000/s → **0.146%**.
10	Device registers occupy addresses in the same address space as memory, so ordinary memory instructions reach them.
11	Finish current instruction → acknowledge and get vector → save PC and flags → run the ISR → restore state and return.

Next: Chapter 13 — Master Formula Sheet and Mixed Practice.

AMJESH singhamjesh.github.io

CHAPTER 13

Master Formula Sheet & Mixed Practice

Every formula of Chapters 2–12 in one place · exam traps · 12 mixed questions with answers

QUICK TRICK

Read this sheet the night before. Each row points to its chapter; reopen the chapter if a row looks unfamiliar.

13.1 Formula sheet

● **LEVEL 2 · EXAM**

Topic (Ch)	Formula / fact
Performance (2)	$T = IC \times CPI / f$ · $CPI = \Sigma\, F_i CPI_i$ · $MIPS = f/(CPI \times 10^6)$ · speedup = T_{old}/T_{new}
Amdahl (2)	$S = 1 / [(1 - f) + f/k]$ · $\max = 1/(1 - f)$
Integers (3)	2's complement $-2^{n-1} \ldots 2^{n-1} - 1$ · negate = invert + 1 · $V = C_{in,MSB} \oplus C_{out,MSB}$
IEEE 754 (3)	single 1|8|23 bias 127 · double 1|11|52 bias 1023 · $(-1)^s \times 1.f \times 2^{e-bias}$ · e = 0 denormal/zero · e = max inf/NaN
Instruction (4)	opcode bits = $\lceil \log_2 N \rceil$ · expanding opcode: (left) × 2^{field} · (A+B)(C+D): 3/6/7/8 instructions for 3/2/1/0-address
Addressing (5)	Direct EA = A · Indirect EA = M[A] · Indexed A + R · PC-relative = PC_{next} + offset · memory refs: imm 0, direct 1, indirect 2
ALU (6)	FA: $S = A \oplus B \oplus C$, $C_{out} = AB + C(A \oplus B)$ · CLA $C_{i+1} = G_i + P_i C_i$ · Booth 10 sub / 01 add · product 2n bits
Control (7)	horizontal width = signals (+next address) · vertical $\Sigma\lceil \log_2(n_i + 1) \rceil$ · store bits = words × width
Pipeline (8)	cycles = $k + n - 1$ · speedup = $nk/(k + n - 1) \le k$ · clock = max stage · CPI = 1 + stalls · RAW: 2 / 0 / 1 stalls
Memory perf. (9)	AMAT = $H + m \cdot P$ · 2-level $H_1 + m_1(H_2 + m_2 P)$ · parallel $h \cdot t_c + (1-h) t_m$ · hierarchical + t_c on miss
Interfacing (9)	chips = (locs ratio) × (width ratio) · address lines = $\log_2$(locs) · low-order interleave
Cache (10)	offset = $\log_2 B$ · sets = lines/k · tag = A − index − offset · bits = lines × (data + tag + valid [+ dirty])
Disk (11)	access = seek + ½(60/RPM) + transfer · capacity = surf × tracks × sect × bytes
I/O (12)	overhead = events/s × cycles / f · DMA: 1 interrupt per block · cycle stealing vs burst

13.2 Ten traps that cost marks

COMMON MISTAKE

- Speedup of a pipeline can never exceed k; the clock is the **slowest** stage plus latch.
- Load-use needs a stall (1) even with forwarding.
- Tag bits come from the address width minus index minus offset; fully associative has no index.
- Total cache size must include tag, valid and (write-back) dirty bits.
- PC-relative uses the **incremented** PC.
- 2's complement range is asymmetric; carry out ≠ signed overflow.
- IEEE 754 has a hidden leading 1; bias is 127 (single) / 1023 (double).
- Amdahl uses fractions of the **original** time.
- Simultaneous vs hierarchical access formulas differ: read the question.
- DMA: one interrupt per **block**, but the CPU still sets it up.

13.3 Mixed practice

Practice questions

● LEVEL 2 · EXAM

Q1. A program has 5×10^8 instructions, average CPI 2.2, on a 1.1 GHz CPU. Execution time?

Q2. Encode −0.15625 in IEEE single precision (hex).

Q3. Addressing: M[100] = 200, M[200] = 300, R1 = 100. What does "LOAD (100)" (indirect) give? And indexed 100(R1)?

Q4. A 5-stage pipeline executes 200 instructions with no stalls. Cycles and speedup over a non-pipelined machine (5 cycles each)?

Q5. Chips needed for a 32K × 16 memory from 8K × 8 chips.

● LEVEL 3 · GATE

Q6. 32-bit address, 8 KB 2-way cache, 16 B blocks. Index and tag bits?

Q7. Instruction mix 40% ALU (1 cycle), 30% load (2 cycles + 1 stall per 4 loads on average), 30% branch (2 cycles). Average CPI?

Q8. LW R1,0(R2); ADD R3,R1,R4 on a pipeline with forwarding. Stall cycles?

Q9. Microprogrammed control store: 1024 words, 36 control signals (horizontal) plus a next-address field. Width and size?

Q10. Cache: 95% hit, hit time 2 ns, memory 100 ns (hierarchical access). Average access time?

Q11. Disk: 7200 RPM, average seek 8 ms, 4 KB read with 0.5 ms transfer. Access time?

Q12. A device sends 1 MB/s in 4-byte words using cycle-stealing DMA with a 200 ns memory cycle. CPU time lost?

Answers

Q	Solution
1	$5\times10^8 \times 2.2 / 1.1\times10^9$ = **1 s**.
2	**0xBE200000** (1.01×2^{-3}, exponent 124).
3	Indirect: M[M[100]] = **300**. Indexed: EA = 200 → **300**.
4	5 + 199 = **204 cycles**. Speedup = 1000/204 ≈ **4.9**.
5	(32/8) × (16/8) = 4 × 2 = **8**.
6	Lines = 512, sets = 256 → index **8**; offset 4; tag = 32 − 12 = **20**.
7	Load effective = 2 + 0.25 = 2.25. CPI = 0.4 × 1 + 0.3 × 2.25 + 0.3 × 2 = 0.4 + 0.675 + 0.6 = **1.675**.
8	**1** (load-use).
9	Address 10 bits → width 36 + 10 = 46 bits. Size = 1024 × 46 = **47,104 bits** (5.75 KB).
10	0.95 × 2 + 0.05 × (2 + 100) = 1.9 + 5.1 = **7.0 ns**.
11	8 + 4.17 + 0.5 = **12.67 ms**.
12	Words/s = 250,000; 250,000 × 200 ns = 50 ms/s = **5%**.

Done. Re-attempt the mixed practice after two days, then solve previous-year GATE questions chapter by chapter.